COMANDO DA AERONÁUTICA · PROJETO SISI

Painel Unificado SISI

Instalação no servidor interno — MySQL / PHP
Instruções para a equipe de TI

Conteúdo do pacote

- **Parte 1 — Checklist de instalação** (requisitos, passos, verificação, operação, problemas)
- **Parte 2 — Guia completo** (arquitetura, configuração, Apache/IIS/Nginx, migração, modelo de segurança, matriz de acesso, rotas, banco, validação, referências)
- **Parte 3 — Telas** do novo acesso (entrada, troca obrigatória, Área Admin)
- **Arquivos:** pasta `sisi/` (index.html, api.php, config.exemplo.php, schema.sql, .htaccess, web.config, vendor/, uploads/), `servidores/nginx_sisi.conf`, `teste_api.sh`, `testes/`

Emitido em 10/10/2026 · pacote u2.30-srv · validado em MariaDB 10.11 + PHP 8.3 + Apache 2.4 + Chromium.
Ao concluir, informar ao gestor do painel: o endereço final e o **token de instalação** definido no config.php (por canal seguro).

Parte 1 — Checklist de instalação

Pacote **u2.30-srv** · API **sisi-api-mysql 2.0** · 10/10/2026

Uma página HTML, uma API em PHP e um banco MySQL/MariaDB. Depois de instalado, o painel **não usa a internet**: bibliotecas e fontes vêm da pasta `vendor/`.

O que mudou desde o pacote u2.28 (os seis pedidos da TI)

Pedido	Como ficou
1. Inicialização e tokens	Aberto pelo endereço da intranet, o painel encontra sozinho a `api.php` da mesma pasta. **Não existe mais token compartilhado nos navegadores**: cada pessoa entra com usuário e senha, e a sessão fica num cookie `HttpOnly` + `SameSite=Strict` de 8 h.
2. Controle de acesso no PHP	A senha é conferida na API (Argon2id/bcrypt). Gestor: tudo. ODS: só **consulta** as OMs do seu ODS. OM: lê e grava só a própria OM. Há limite de tentativas e troca obrigatória da senha no primeiro acesso.
3. Leituras e imagens	Toda leitura exige sessão e respeita o escopo. Fotos só pela `api.php?rota=foto`, com nome, escopo e tipo de imagem conferidos. A pasta `uploads/` fica bloqueada para acesso direto.
4. Bibliotecas locais	xlsx, papaparse, chart.js, html2canvas, jspdf e as fontes Roboto em `sisi/vendor/`, com hashes em `vendor/LEIA-ME.txt`.
5. SQL, usuário e config	`schema.sql` com 12 tabelas e o rol de 69 OMs; usuário `sisi_app` com privilégio mínimo; `config.exemplo.php` revisto.
6. Teste completo	125 verificações da API, 39 do servidor e 49 no navegador (gestor e duas OMs em navegadores separados). Ver "Validação" no README.

Requisitos

- Apache 2.4, Nginx ou IIS 10, com **PHP 7.4 ou superior** (testado em 8.3) e as extensões `pdo_mysql` e `fileinfo` (esta é recomendada; sem ela a API confere a assinatura do arquivo).
- MySQL 5.7+/8.x ou MariaDB 10.3+ (phpMyAdmin é suficiente).
- `post_max_size` e `upload_max_filesize` de pelo menos **64 MB**.
- **HTTPS com certificado interno é fortemente recomendado**: com `http://`, a senha trafega sem cifragem na rede.

Instalação (cerca de 20 minutos)

1. **Banco.** No phpMyAdmin, aba Importar, envie `sisi/schema.sql`. Ele cria o banco `sisi` com 12 tabelas e o rol de OMs. Pode ser reimportado sem perder dados.
2. **Usuário do banco.** Crie `sisi_app` com senha forte e **somente** `SELECT, INSERT, UPDATE, DELETE` no banco `sisi`. O comando está comentado no fim do `schema.sql`.
3. **Configuração.** Copie `sisi/config.exemplo.php` para `config.php` e preencha:
   - `db_host`, `db_user`, `db_pass`;
   - `token`: um valor aleatório longo (`openssl rand -hex 24`). Serve **só** para criar o gestor inicial. Entregue-o ao gestor por canal seguro;
   - `cookie_seguro => true` se o acesso for por `https://`.
4. **Arquivos.** Publique a pasta `sisi/` inteira (ex.: `/var/www/html/sisi/`). Permissões no Linux: `uploads/` com escrita para o usuário do servidor web (`chown www-data uploads; chmod 770`); `config.php` com `chown root:www-data` e `chmod 640`.
   - **Apache:** o `.htaccess` já bloqueia tudo. Exige `AllowOverride All` na pasta, e `mod_headers` é recomendado.
   - **IIS:** o `web.config` da pasta e o de `uploads/` fazem o mesmo.
   - **Nginx:** inclua `servidores/nginx_sisi.conf` no bloco `server`.
   - Recomendado: mover `uploads/` para fora da pasta pública (chave `uploads` do `config.php`).
5. **Verificação.** No navegador:
   - `.../sisi/api.php?ping=1` deve responder `{"ok":true,"versao":"sisi-api-mysql 2.0",...}`;

- `.../sisi/config.php` e `.../sisi/uploads/` devem responder **403** (ou 404);
- `.../sisi/index.html` deve abrir a tela de entrada com **Usuário** e **Senha**.

Num Linux com `curl`, rode também `bash testes/teste_servidor.sh http://<servidor>/sisi`. São 39 conferências, todas somente leitura.

6. **Primeiro acesso.** O gestor abre o painel, clica em **Primeira configuração** e informa o token, o próprio usuário e a senha. Em seguida, na aba Sincronização da Área Admin, usa **Importar do JSONBin** (ou um arquivo `.json`) para trazer os dados e os usuários atuais.
7. **Informe ao gestor** o endereço final (ex.: `https://intranet/sisi/`).

Operação

- **Backup semanal:** exporte o banco `sisi` (SQL) e copie a pasta `uploads/`.
- **Segurança:** a tabela `seguranca_log` registra logins, falhas, bloqueios, acessos negados e alterações de usuários. A tabela `api_log` registra as chamadas. As duas podem ser esvaziadas periodicamente.
- **Usuário bloqueado** (5 senhas erradas do mesmo computador, ou 20 de computadores diferentes, em 15 min): espera de 15 minutos, ou o gestor gera uma nova senha na Área Admin.
- **Proxy reverso ou NAT:** se todos os acessos chegarem à API com o mesmo IP, informe o IP do proxy em `proxies_confiaveis`. Sem isso, o limite de 30 falhas por IP vale para todos juntos.
- **Gestor esqueceu a senha:** outro gestor gera uma nova. Se não houver outro, a TI desativa os gestores (`UPDATE usuarios SET ativo=0 WHERE perfil='gestor'`) e o painel volta a oferecer a "Primeira configuração", que exige o `token`. Usando o mesmo usuário, o cadastro é reativado com a nova senha.

Se algo falhar

Sintoma	Causa provável
`config.php ausente`	arquivo não copiado/renomeado
`banco indisponível`	`db_host`, `db_user`, `db_pass`; o detalhe do erro fica no log de erros do PHP
login sempre "sessão ausente ou expirada"	`cookie_seguro => true` com acesso por `http://` (o navegador não devolve o cookie)
`token anti-CSRF ausente`	página aberta antes de uma atualização: recarregue (Ctrl+F5)
`origem da requisição não autorizada`	painel aberto por outro host; deixe painel e API no mesmo endereço ou preencha `cors_origin`
`não foi possível criar a pasta de fotos`	`uploads/` sem permissão de escrita
fonte ou gráfico não aparecem	a pasta `vendor/` não foi publicada
500 sem mensagem	PHP sem `pdo_mysql`, ou versão anterior à 7.4

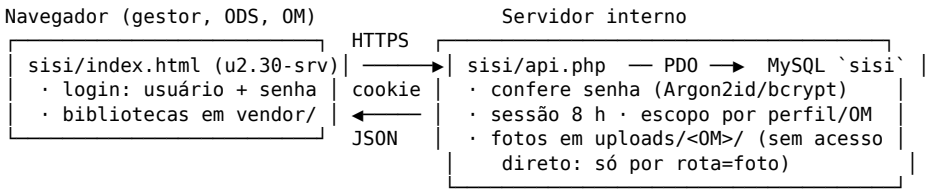
O guia completo (arquitetura, matriz de acesso, Nginx e IIS, migração, consultas SQL e referências) está em `README_IMPLANTACAO.md`.

Parte 2 — Guia completo de implantação

Pacote **u2.30-srv** · API **sisi-api-mysql 2.0** · 10/10/2026 Checklist resumido: `LEIA-ME_TI.md`. Este guia traz os detalhes.

1. Visão geral

O painel publicado no GitHub Pages (linha principal **u2.30**) usa três serviços na internet: jsonbin.io (estado do gestor), Google Apps Script/Drive (pacotes e fotos das OMs) e uma API opcional. Este pacote substitui os três por uma **API em PHP com banco MySQL/MariaDB** dentro da rede da OM.



A variante é gerada da linha principal pelo script `para_o_gestor/build_u230_variante_srv.py` (53 alterações por âncora). O `index.html` do GitHub **não é alterado**. Aberta como arquivo local, sem servidor, a variante se comporta como a u2.30.

2. O que vai para o servidor

```
sisi/                          ← pasta pública (ex.: /var/www/html/sisi)
├── index.html                 ← painel u2.30-srv
├── api.php                    ← a API (única página PHP executável)
├── config.php                 ← criar a partir de config.exemplo.php (NÃO versionar)
├── config.exemplo.php
├── schema.sql                 ← importar uma vez (pode reimportar)
├── .htaccess                  ← regras do Apache
├── web.config                 ← regras do IIS
├── vendor/                    ← bibliotecas JS e fontes locais (LEIA-ME.txt com SHA-256)
└── uploads/                   ← fotos (escrita do servidor web; sem acesso direto)
    ├── .htaccess
    └── web.config
servidores/nginx_sisi.conf     ← regras equivalentes para o Nginx
testes/                        ← verificação do servidor, teste no navegador, roteador do php -S
teste_api.sh                   ← 125 verificações da API num MariaDB temporário
```

Não precisam ir para o servidor: `schema.sql` e `config.exemplo.php` (as regras os bloqueiam, de qualquer forma), `testes/`, `docs/` e `para_o_gestor/`.

3. Pré-requisitos

Item	Mínimo	Observação
PHP	7.4 (testado em 8.3.6)	`pdo_mysql` obrigatória; `fileinfo` recomendada; `mbstring` e `iconv` opcionais
Banco	MySQL 5.7+/8.x ou MariaDB 10.3+	testado em MariaDB 10.11.14
Servidor web	Apache 2.4 (testado em 2.4.58), Nginx ou IIS 10	Apache: `AllowOverride All` e `mod_headers`
Upload	`post_max_size` e `upload_max_filesize` ≥ 64 MB	o `.htaccess` tenta 80 MB quando o PHP roda como módulo
HTTPS	recomendado	sem ele, senha e cookie trafegam sem cifragem na rede

4. Passo a passo

4.1 Banco de dados

1. phpMyAdmin → **Importar** → `schema.sql` → Executar (ou `mysql -u root -p < schema.sql`). Cria o banco `sisi` com 12 tabelas e o rol inicial de 69 OMs.
2. Usuário da aplicação, com privilégio mínimo (o comando está no fim do `schema.sql`):

```
CREATE USER 'sisi_app'@'localhost' IDENTIFIED BY '<senha forte>';
GRANT SELECT, INSERT, UPDATE, DELETE ON `sisi`.* TO 'sisi_app'@'localhost';
```

A API não precisa de `CREATE`, `DROP`, `ALTER` nem `GRANT`. Se o MySQL estiver em outra máquina, troque `localhost` pelo IP do servidor web.

4.2 `config.php`

Chave	Padrão	Para que serve
`db_host`, `db_port`, `db_name`, `db_user`, `db_pass`	`localhost`, `3306`, `sisi`, `sisi_app`	conexão PDO
`token`	—	**token de instalação**: só cria o gestor inicial quando não há gestor ativo. Valor longo e aleatório (`openssl rand -hex 24`), com pelo menos 16 caracteres
`sessao_horas`	8	validade absoluta da sessão (decisão do gestor)
`cookie_nome`	`SISI_SESS`	nome do cookie de sessão
`cookie_seguro`	`false`	`true` quando o acesso for por `https://`
`senha_min`	12	mínimo de caracteres da senha escolhida pelo usuário (decisão do gestor)
`login_max_falhas` / `ip_max_falhas` / `login_janela_min`	5 / 30 / 15	bloqueio temporário por usuário e por IP
`cors_origin`	vazio	só se o painel ficar em **outro** host; informe a origem exata. * é ignorado
`proxies_confiaveis`	`[]`	IPs de proxy reverso cujo `X-Forwarded-For` é aceito
`uploads`	`__DIR__.'/uploads'`	pasta das fotos; **recomendado fora da pasta pública**
`max_mb` / `foto_max_mb`	64 / 10	limite do envio e de cada foto
`hist_max`	30	envios anteriores guardados por OM

Para tirar o `config.php` da pasta pública, defina a variável de ambiente `SISI_CONFIG` com o caminho completo. No Apache: `SetEnv SISI_CONFIG /etc/sisi/config.php`. No IIS: FastCGI Settings → Environment Variables.

Permissões no Linux: `chown root:www-data config.php && chmod 640 config.php`; `chown www-data:www-data uploads && chmod 770 uploads`.

4.3 Servidor web

Apache. O `.htaccess` da pasta:

- só deixa executar `api.php` (todo outro `.php` recebe 403);
- nega `.sql`, `.md`, `.log`, `.config` e arquivos ocultos;
- desliga a listagem de pastas;
- envia `Content-Security-Policy`, `X-Frame-Options`, `nosniff` e `Referrer-Policy`, quando há `mod_headers`.

O `uploads/.htaccess` nega tudo. Requer no `httpd.conf`:

```
<Directory /var/www/html/sisi>
    AllowOverride All
</Directory>
```

e `a2enmod headers` (recomendado).

IIS 10. Os arquivos `web.config` (pasta e `uploads/`) usam Request Filtering:

- `hiddenSegments` para `uploads`, `config.php` e `schema.sql`;
- extensões negadas;
- `maxAllowedContentLength` de 80 MB;
- os mesmos cabeçalhos de segurança.

O PHP deve rodar via FastCGI.

Nginx. Inclua `servidores/nginx_sisi.conf` no bloco `server` e ajuste o socket do PHP-FPM. No `php.ini`, defina `post_max_size = 80M`, `upload_max_filesize = 80M` e `expose_php = Off`.

4.4 Verificação

- `.../api.php?ping=1` → `{"ok":true,"versao":"sisi-api-mysql 2.0",...}`
- `.../config.php`, `.../uploads/`, `.../schema.sql` → 403 ou 404
- `bash testes/teste_servidor.sh https://<servidor>/sisi` → 39 conferências somente leitura (arquivos bloqueados, sessão exigida em todas as leituras, cabeçalhos, bibliotecas locais)

5. Primeiro acesso e migração

1. **Gestor inicial.** Na tela de entrada, use **Primeira configuração** e informe o `token`, o usuário (sugestão: e-mail institucional), posto, nome e senha. A opção só aparece enquanto não houver gestor ativo.
2. **Migração.** Área Admin → aba **Sincronização** → Migração dos dados atuais:
   - **Importar do JSONBin:** este computador precisa de internet. Usa o Bin ID do cartão "Compartilhamento · JSONBin" e, se o bin for privado, a chave;
   - **Importar de arquivo .json:** salve antes o conteúdo de `https://api.jsonbin.io/v3/b/<Bin ID>/latest`.

O painel grava o estado no MySQL e importa os usuários. A lista mostra o **login** de cada um, gerado a partir do e-mail ou como `perfil.escopo.sobrenome`, por exemplo `om.bapv.silva`.

3. **Usuários migrados** entram com o login informado pelo gestor e a **senha que já tinham**. A API confere o hash antigo `sha256(sal+senha)`, que no banco fica embrulhado em `password_hash`, e obriga a criar uma senha nova nesse mesmo acesso. A nova vai para `password_hash` e o hash antigo é apagado (decisão do gestor, 10/10/2026).
4. **Pacotes das OMs:** cada OM entra pelo novo endereço e clica **Enviar ao servidor** uma vez, porque os dados estão no navegador dela. As fotos antigas continuam no Drive; se quiser trazê-las, copie para `uploads/<SIGLA>/` e cadastre na tabela `fotos`.

6. Modelo de segurança

6.1 Autenticação e senhas

- **Login por usuário e senha, conferido só no servidor.** Hashes nunca saem do banco, e o estado público não traz mais o campo `auth`.
- **Armazenamento:** `password_hash()` com Argon2id quando o PHP tem suporte, senão bcrypt (`password_needs_rehash` atualiza na próxima entrada).
- **Política:**
- mínimo de 12 caracteres e máximo de 128;
- sem regras de composição;
- bloqueio de senhas comuns, repetitivas, com o nome do sistema ou com o nome/usuário da pessoa.
- **Senhas definidas pelo gestor** (cadastro, "Nova senha", "Reenviar e-mail") são **provisórias**: chegam como `sha256(sal+senha)` gerado no navegador do gestor. A API guarda esse valor embrulhado em `password_hash`, para que não possa ser quebrado rapidamente se o banco vazar e para que o tempo de resposta não revele quem está com senha provisória. Valem só até o primeiro acesso e obrigam a troca.
- **Limite de tentativas** (em 15 minutos → HTTP 429):
- 5 falhas do mesmo usuário vindas do mesmo IP;
- 20 falhas do mesmo usuário de qualquer IP (assim um terceiro não trava a conta de alguém com poucas tentativas);
- 30 falhas por IP.

A conferência é serializada por usuário (`GET_LOCK`): uma rajada de requisições paralelas não burla o limite (teste no Apache: 25 tentativas simultâneas, 5 conferidas e 20 recusadas). A mensagem de erro é a mesma para usuário inexistente e para senha errada, e o tempo de resposta também é equivalente.

6.2 Sessão

- **Decisão técnica:** cookie `HttpOnly` + `SameSite=Strict` (+ `Secure` com HTTPS), e não `Authorization: Bearer` guardado pelo JavaScript.
- O token fica fora do alcance de scripts, o que limita o roubo de sessão por XSS.

- As fotos podem ser exibidas por `<img src>` comum (com Bearer, cada imagem teria de ser baixada por `fetch` e convertida em blob).
- O cookie sem `Expires` some ao fechar o navegador.
- **Valor e armazenamento:** 256 bits aleatórios (`random_bytes`). O banco guarda só o SHA-256 do valor.
- **Validade:** absoluta de 8 h, contada pelo relógio do MySQL. **Sair** apaga a sessão no servidor. Troca de senha derruba as demais sessões do usuário, e redefinição de senha ou remoção pelo gestor derruba todas.
- **Anti-CSRF (defesa em camadas):**
- token por sessão no cabeçalho `X-CSRF-Token` em toda gravação;
- recusa de `Origin` e `Sec-Fetch-Site` de outro site;
- `SameSite=Strict`.

6.3 Matriz de autorização (conferida em toda requisição)

Recurso	Gestor	ODS	OM
Estado (execução, prazos, avisos) — ler	✔	✔	✔
Estado — gravar	✔	✘ 403	✘ 403
Pacote/dados de OM (`?om=`, `rota=pacotes`, `?all=1`) — ler	todas	OMs do seu ODS	só a própria
Pacote de OM — enviar/gravar	✔	✘ 403 (somente consulta)	só a própria
Fotos (`rota=foto`)	todas	OMs do seu ODS	só a própria
Auditoria (`rota=auditoria`)	todas	OMs do seu ODS	só a própria
Ciências — registrar	a própria	a própria	a própria
Ciências — consultar	todas	do ODS e das suas OMs	só a própria
Rol de OMs — ler / gravar	✔ / ✔	✔ / ✘	✔ / ✘
Usuários (`rota=usuarios`)	✔	✘	✘

- **A qual ODS uma OM pertence** é decidido pela tabela `roster`, editável só pelo gestor, e **nunca pelo que o navegador envia**.
- No envio do pacote, `om`, `omNome`, `ods` e "enviado por" são reescritos pela API a partir do rol e da sessão.

6.4 Fotos

- **Nome do arquivo:** `^[A-Za-z0-9][A-Za-z0-9._-]*\.(jpe?g|png|webp)$`, sem `..`, com até 180 caracteres.
- **Tipo:** detectado pelo **conteúdo** (`finfo` ou assinatura do arquivo). Arquivo que não é JPEG, PNG ou WebP é recusado. Cada foto pode ter até 10 MB.
- **Entrega:** só por `rota=foto`, com:
- sessão, escopo e registro na tabela `fotos`;
- caminho conferido com `realpath` dentro de `uploads`;
- `Content-Type` correto, `nosniff`, `Cache-Control: private` e `Content-Security-Policy: sandbox`.
- **Mesmo nome com outra imagem** (foto trocada, ou registro novo com o mesmo título e data): o arquivo antigo nunca é sobrescrito. A nova foto ganha um sufixo do conteúdo (`_a1b2c3d4`), e a referência no pacote é corrigida pela API.
- **Acesso direto a `uploads/`:** negado (Apache, IIS e Nginx), assim como qualquer cópia do `config.php` (`config.php~`, `.bak`, `.save`, `.txt`).

6.5 Outras medidas

- **Cabeçalhos da API:** `nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, CSP `default-src 'none'`, `Cross-Origin-Resource-Policy: same-origin`. Sem CORS por padrão.
- **CSP do painel:** só a própria origem (exceção: `api.jsonbin.io`, usada pela migração). `'unsafe-inline'` continua necessário porque o painel é um único HTML com scripts embutidos.
- **Erros de banco** aparecem só no log do PHP; o navegador recebe mensagem genérica.
- **Registros:** `seguranca_log` (eventos de segurança) e `api_log` (chamadas). Na `auditoria`, os campos do evento são declarados pelo painel da OM. A API acrescenta `enviado_por_login` (o usuário da sessão que enviou) e `recebido_em` (relógio do servidor), que não podem ser forjados.
- **Cadastro de usuários:** nome, posto, telefone e e-mail não aceitam `< > " \`` . A gravação é serializada, e a versão da lista é calculada pelo conteúdo: duas abas ou um duplo clique não duplicam nem sobrescrevem cadastros.
- **Cookie** `Secure` **automático** quando a requisição chega por HTTPS, mesmo com `cookie_seguro` em `false`.
- **Ao sair:** o painel apaga deste navegador o consolidado das OMs e a lista de usuários. O rascunho não enviado da OM é mantido para não perder trabalho.

7. Rotas da API

Rota	Método	Acesso
`?ping=1`	GET	público (sem versões de software)
`?rota=sessao`	GET	público: quem está logado, `csrf`, se já foi instalado
`?rota=login` · `?rota=logout`	POST	público (com limite de tentativas) · sessão
`?rota=senha`	POST	sessão (também durante a troca obrigatória)
`?rota=instalar`	POST	`token` do config.php, só sem gestor ativo
`?rota=usuarios`	GET · POST `sincronizar/importar`	gestor
`?rota=estado`	GET · POST/PUT	sessão · gestor
`?roster=1`	GET · POST	sessão · gestor
`?om=X`	GET · POST	escopo de leitura · escopo de gravação
`?all=1` · `?rota=pacotes[&om=][&lista=1]`	GET	escopo de leitura
POST com `{om, dados, fotos}`	POST	escopo de gravação
`?rota=ciencia` · `?rota=ciencias`	POST · GET	sessão · escopo
`?rota=auditoria[&om=]` · `?rota=foto&om=&nome=`	GET	escopo de leitura

Os formatos de resposta são os mesmos da versão 1.0 (contratos JSONBin, Apps Script e API própria). Gravações exigem `X-CSRF-Token`.

8. Banco de dados

Tabela	Conteúdo
`estado`	estado do gestor (JSON, sem credenciais)
`pacotes` / `pacotes_hist`	versão vigente de cada OM / últimos 30 envios
`fotos`	índice das fotos (arquivo em `uploads/<OM>/`)
`auditoria`	eventos de cada OM (quem entrou, o que alterou)
`ciencias`	ciência dos avisos
`roster`	rol de OMs, a fonte do escopo ODS
`usuarios`	usuários, perfil, escopo, hash da senha
`sessoes`	sessões ativas (SHA-256 do cookie)
`login_tentativas`	tentativas de login (limpeza automática após 30 dias)

Tabela	Conteúdo
`seguranca_log` / `api_log`	eventos de segurança / chamadas

Consultas úteis:

```
-- quem está com troca de senha pendente e quem nunca entrou
SELECT login, nome, perfil, escopo, troca_obrigatoria, ultimo_login FROM usuarios ORDER BY perfil, escopo;
-- acessos negados e falhas de login (últimos 7 dias)
SELECT quando, evento, login, ip, detalhe FROM seguranca_log
 WHERE evento IN ('login_falha','login_bloqueado','negado') AND quando > NOW() - INTERVAL 7 DAY ORDER BY id DESC;
-- sessões abertas agora
SELECT u.login, s.criada_em, s.expira_em, s.ip FROM sessoes s JOIN usuarios u ON u.id = s.usuario_id WHERE s.expira_em > NOW
-- derrubar todas as sessões (ex.: após incidente)
DELETE FROM sessoes;
-- última atualização de cada OM
SELECT om, ods, saved_at, enviado_por, fotos FROM pacotes ORDER BY saved_at DESC;
```

9. Operação

- **Backup semanal:** exporte o banco `sisi` e copie `uploads/`. Teste a restauração ao menos uma vez.
- **Limpeza** (opcional, mensal):

```
DELETE FROM api_log WHERE quando < NOW() - INTERVAL 90 DAY;
DELETE FROM seguranca_log WHERE quando < NOW() - INTERVAL 1 YEAR;
DELETE FROM sessoes WHERE expira_em < NOW();
```

- **Atualizar o painel:** rode `para_o_gestor/build_u230_variante_srv.py` sobre a nova linha principal e publique o `sisi/index.html` gerado. Se uma âncora mudou, o script para e diz qual.

10. Validação realizada (10/10/2026)

Ambiente: Ubuntu 24.04, PHP 8.3.6 (mod_php), Apache 2.4.58 com o `.htaccess` deste pacote, MariaDB 10.11.14, Chromium (Playwright 1.56). Os registros completos estão em `docs/`.

Bateria	Resultado	O que cobre
`teste_api.sh`	**125/125**	rajada paralela de logins, hash provisório embrulhado, tempo de resposta igual, foto com nome repetido, e-mail com '+' no login, HTML no nome, gravações simultâneas, origem exata; instalação, política de senha, troca obrigatória, migração do hash antigo, limite de tentativas, CSRF, origem, matriz gestor/ODS/OM, fotos (tipo real, nome malicioso, `../`), sessão expirada, logout, redefinição e remoção de usuário, registros de segurança
`testes/teste_servidor.sh` (Apache real)	**39/39**	bloqueio de `config.php`, `uploads/`, `.sql`, `.htaccess` e `web.config`; 401 em todas as leituras sem sessão; cabeçalhos; bibliotecas locais
`testes/e2e_navegadores.cjs`	**49/49**	gestor, OM A e OM B em navegadores separados com a **internet bloqueada**: detecção automática, instalação, migração por arquivo, cadastro pela Área Admin, primeiro acesso com troca, envio com foto, recuperação noutro navegador, fotos exibidas, tentativas entre OMs, perfil ODS, sessão de 8 h vencida, saída, nenhuma violação de CSP, nenhum erro de JavaScript
Modo arquivo local (sem servidor)	ok	variante sem a API volta ao comportamento da u2.30

Não testado aqui: IIS e Nginx reais (os arquivos seguem a documentação de cada servidor, mas precisam ser conferidos com `teste_servidor.sh` após a instalação), MySQL 8 (testado em MariaDB), Windows, HTTPS com certificado interno.

11. Pontos de atenção e recomendações

1. **HTTPS.** Use certificado interno e `cookie_seguro => true`. É a medida isolada de maior impacto que falta.
2. **xlsx 0.18.5.** É a versão que a u2.30 usa (mantida para não mudar o comportamento). Ela tem duas vulnerabilidades publicadas, ambas ao **ler** planilhas maliciosas: **CVE-2023-30533** (poluição de protótipo, corrigida na 0.19.3) e **CVE-2024-22363** (ReDoS, corrigida na 0.20.2). As versões corrigidas não estão no npm: a SheetJS as distribui em cdn.sheetjs.com, e os avisos oficiais também estão lá (`/advisories/`). O risco aqui é baixo, porque só o gestor importa planilhas, vindas de fontes oficiais. Mesmo assim, recomenda-se atualizar a linha principal para a versão corrigida e regenerar a

variante.

3. **Segundo fator (MFA)** não foi implementado. Para perfis de gestor, é a evolução natural (ex.: TOTP), conforme as boas práticas citadas abaixo.
4. **Inatividade.** A sessão vale 8 h corridas (decisão do gestor), sem encerramento por inatividade.
5. **Estado do gestor visível a todos os perfis.** A `rota=estado` (execução financeira, prazos, RaP e planejamento) é lida por qualquer usuário logado, como já acontecia com o JSONBin; é o painel que filtra a tela por OM e ODS. Assim, uma OM tecnicamente consegue ler os números financeiros das demais. Os pacotes, fotos, auditoria e ciências já são filtrados no servidor. Filtrar também o estado é possível (por linhas de cada OM), mas muda a forma como o painel carrega os dados: **depende de decisão do gestor**.
6. **Computador compartilhado.** Ao sair, o consolidado é apagado do navegador, mas o rascunho não enviado da OM permanece (para não perder trabalho). Oriente o uso de contas individuais do sistema operacional.

12. Referências consultadas

- **OWASP** Application Security Verification Standard (ASVS) 4.0: capítulos V2 autenticação, V3 sessão, V4 controle de acesso, V12 arquivos e V14 configuração. OWASP Cheat Sheet Series: Password Storage (Argon2id; bcrypt com custo ≥ 10 e limite de 72 bytes), Session Management, Cross-Site Request Forgery Prevention, File Upload, Authentication. Fonte: owasp.org / cheatsheetseries.owasp.org.
- **NIST SP 800-63B** (Digital Identity Guidelines: autenticação):
- tamanho mínimo, sem regras de composição;
- lista de senhas proibidas;
- limite de tentativas.

A revisão 4 (2025) pede 15 caracteres quando a senha é o único fator. Este dado foi conferido em fontes secundárias, porque o site do NIST não estava acessível deste ambiente. O gestor optou por 12, configurável em `senha_min`.

- **Brasil:**
- **Guia de Requisitos Mínimos de Privacidade e Segurança da Informação para Aplicações Web** (Programa de Privacidade e Segurança da Informação, PPSI, da Secretaria de Governo Digital), em gov.br/governodigital. O documento existe, mas o conteúdo dos itens não pôde ser aberto deste ambiente; recomenda-se que a TI faça a conferência item a item com o guia oficial;
- **Lei nº 13.709/2018 (LGPD), art. 46**: medidas de segurança técnicas e administrativas para proteger dados pessoais (os cadastros de usuários têm nome, e-mail e telefone).
- **PHP Manual:** `password_hash`, `password_verify`, `random_bytes`, `setcookie` (opções `httponly` e `samesite`).

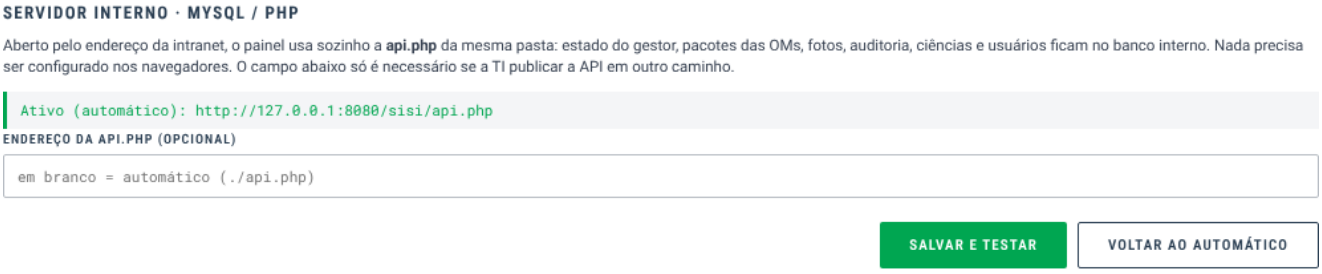
Parte 3 — Telas do novo acesso



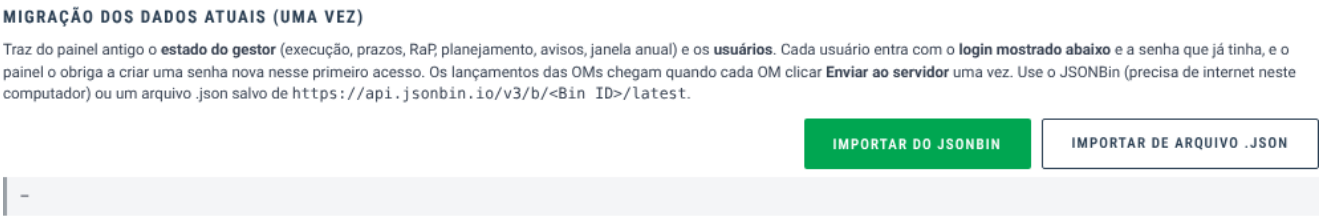
Entrada: usuário e senha (o perfil é reconhecido pela API).



Primeiro acesso: criação obrigatória da senha pessoal.



Área Admin → Sincronização: servidor detectado automaticamente.



Migração única do estado e dos usuários do JSONBin.



Usuários: login, situação da senha e último acesso.